★ SYSTEM DESIGN HANDBOOK ★
PART - 2

1. DATABASE INDEXING

What is Index ?

→ Index is a data structure that improves the speed of data retrieval operations on a database table.

Example

Table : users

id	name	age	city
1	Alice	25	Delhi
2	Bob	30	Mumbai
3	Charlie	28	Pune
4	David	35	Delhi
.	.	.	.

↓

Index on (city)

city	pointer (id)
Delhi	→ [1, 4]
Mumbai	→ [2]
Pune	→ [3]

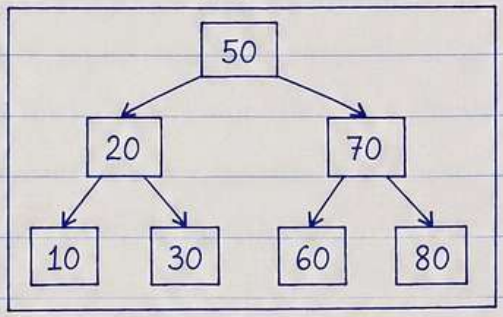
Why do we need Index ?

- Without index → DB has to scan the entire table (Full Table Scan).
- With index → DB can quickly find the data using the index structure.
- Index = Faster Reads
  (Some extra space + write cost)

Types of Index

1. Clustered Index → Actual data is stored in the same order as the index.
   Only one clustered index per table.
2. Non-Clustered Index → Index is separate from actual data. Contains a pointer / reference to the actual data.
   Multiple non-clustered indexes can exist on a table.
3. Composite Index → Index on multiple columns.
4. Unique Index → Ensures all values in the indexed column are unique.
5. Full Text Index → Used for full text search (like searching in articles).

B-Tree (Simplified)



B-Tree keeps data sorted and balanced which makes search, insert and delete operations very efficient → O (log n)

Key Points

- Index improves read performance.
- Too many indexes can slow down write operations.
- Choose columns with high selectivity for indexing.
- Indexes take extra storage space.

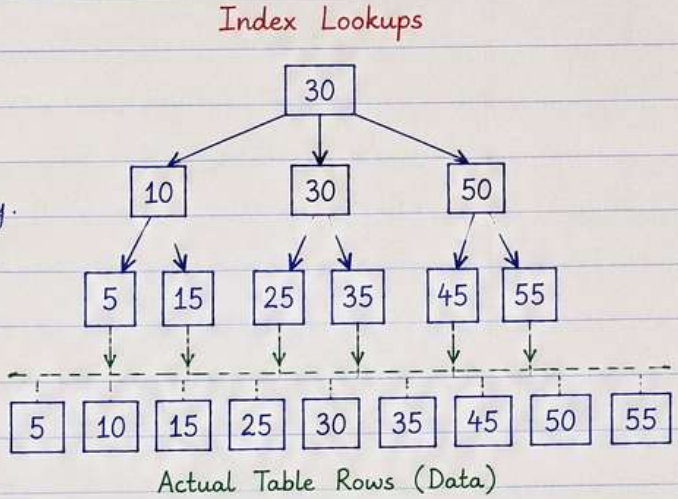
1. DATABASE INDEXING (CONTINUED)

How Index Works ?

→ Index is like the index of a book.

→ Instead of scanning the whole table, DB uses index to find the data quickly.

→ Index stores a small pointer to the actual data location.



Example :

select * from users where id = 30;

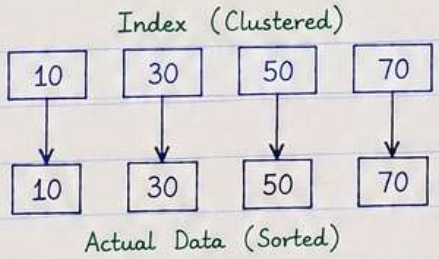
Without Index → DB scans all rows one by one.

With Index → DB directly jumps to 30 using index.

Types of Index

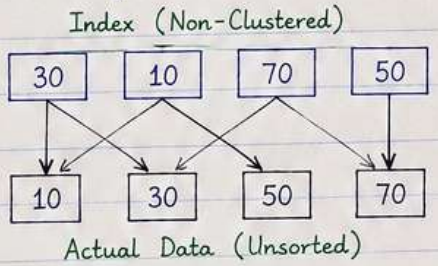
1. Clustered Index

- Actual data stored in the same order as the index.
- Only one clustered index per table.
- Fast for range queries.
- Changing order is expensive.



2. Non-Clustered Index

- Index is separate from actual data.
- Contains pointer / reference to the actual data.
- Multiple non-clustered indexes allowed.
- Extra storage required.



3. Composite Index

- Index on multiple columns.
- Useful when queries filter on more than one column.
- Order of columns in index matters.

Example :

Index on (name, age)

Query → where name = 'Alice' and age = 25

4. Covering Index

- Index includes all columns required by the query.
- DB can return result using only the index (no need to read actual data).
- Improves performance a lot.

When to Use Index ?

- Columns used in WHERE clause.
- Columns used in JOIN.
- Columns used in ORDER BY / GROUP BY.
- Columns with high selectivity (unique / few distinct values).
- Avoid indexing columns with low selectivity (like gender, status, etc.).

Remember

More indexes = faster reads but slower writes.

☺

2. DATABASE TRANSACTIONS

What is Transaction?

→ Transaction is a sequence of one or more SQL operations that are executed as a single unit of work.

→ Either all operations are successful (COMMIT) or none of them are (ROLLBACK).

Example : Bank Transfer

Two operations must happen together :

1. Debit ₹200 from A
2. Credit ₹200 to B

Either both happen or none happen.

ACID Properties

A - Atomicity → All or Nothing

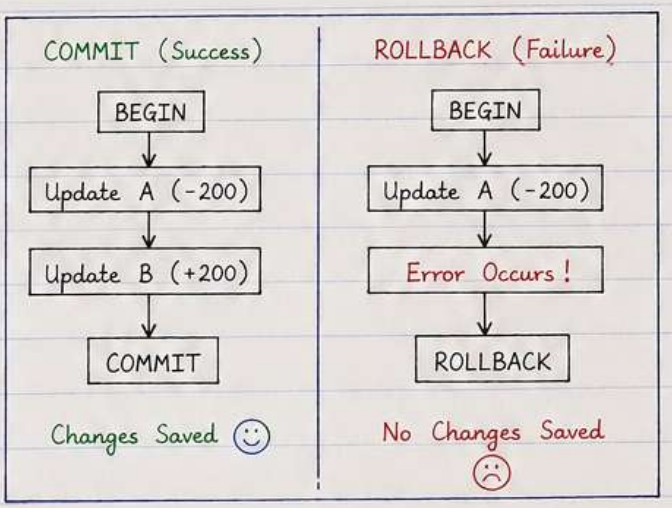
C - Consistency → DB remains in a valid state

I - Isolation → Transactions do not affect each other

D - Durability → Once committed, data is permanent

Transaction Flow

1. Begin Transaction
   → DB starts a transaction.
2. Perform Operations
   → Execute one or more queries (INSERT, UPDATE, DELETE).
3. Commit / Rollback
   → Commit → save changes permanently.
   → Rollback → undo all changes.



SQL Example

Transfer ₹200 from A to B

```
BEGIN TRANSACTION;
UPDATE accounts SET balance = balance - 200
  WHERE id = 1;   -- Debit from A
UPDATE accounts SET balance = balance + 200
  WHERE id = 2;   -- Credit to B
COMMIT;           -- If everything is fine
```

If something goes wrong

```
BEGIN TRANSACTION;
UPDATE accounts SET balance = balance - 200
  WHERE id = 1;
-- Some error happens
ROLLBACK;   -- Undo all changes
```

Isolation Levels (Brief)

- Read Uncommitted → Can read uncommitted changes (Dirty Read possible)
- Read Committed → Can read only committed changes
- Repeatable Read → Same query result in a transaction
- Serializable → Highest level, transactions execute one by one

Use Transactions whenever multiple operations are dependent on each other. ☺

3. PROXY vs REVERSE PROXY

What is Proxy ?

→ A proxy acts as an intermediary between client and server.

→ It receives the request from the client, forwards it to the server and sends the response back.

Why use Proxy ?

- Security (hide real server)
- Access Control
- Caching & Compression
- Load Balancing
- Logging & Monitoring

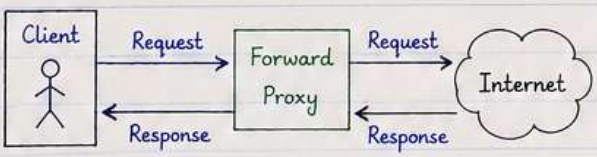
1. FORWARD PROXY (Client Side Proxy)

→ Sits between the client and the internet.

→ Used by clients to access the internet.

→ Helps in anonymity, access control, caching etc.

Diagram :



Use Cases :

1. Access blocked websites.
2. Hide client IP address.
3. Caching to reduce bandwidth.
4. Organization policies & content filtering.

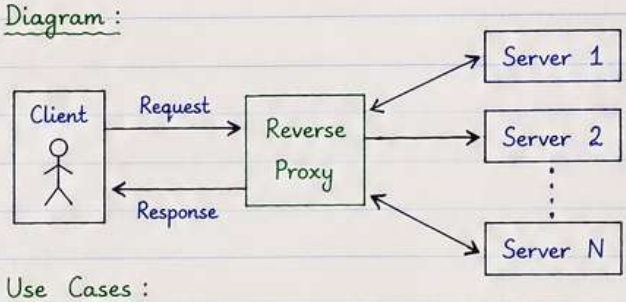
2. REVERSE PROXY (Server Side Proxy)

→ Sits between client and backend server(s).

→ Used by servers to handle client requests.

→ Helps in security, load balancing, SSL termination etc.

Diagram :



Use Cases :

1. Load balancing across multiple servers.
2. Hide internal server details.
3. SSL termination & encryption.
4. Caching static content.

Forward Proxy vs Reverse Proxy

Feature	Forward Proxy	Reverse Proxy
Position	Between client and internet	Between client and server(s)
Used By	Clients	Servers
Purpose	Access control, anonymity, caching	Security, load balancing, caching, SSL termination
Hides	Client IP	Server IP
Example	Browser proxy, corporate proxy	Nginx, HAProxy, Cloudflare

Example (Nginx as Reverse Proxy)

nginx.conf (basic)

```
server {
    listen 80;              # Nginx listens on port 80
    server_name myapp.com;

    location / {
        proxy_pass http://backend_servers;  # forward to backend
    }
}
```

backend_servers (upstream)

```
upstream backend_servers {
    server 10.0.0.1:8080;
    server 10.0.0.2:8080;
    server 10.0.0.3:8080;
}
```

Nginx receives all requests on port 80 and forwards them to backend servers.

4. JWT (JSON WEB TOKEN)

What is JWT ?

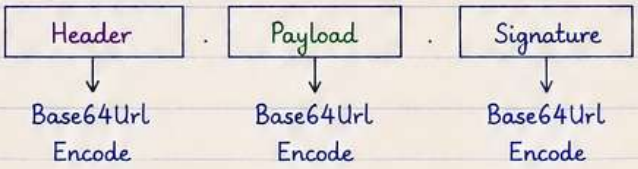
→ JWT is a compact, URL-safe token used to securely transmit information between parties as a JSON object.

→ It is commonly used for Authentication.

Why use JWT ?

- Stateless (No session storage on server)
- Scalable
- Can be used across microservices
- Secure (digital signature)
- Supports expiration

Structure of JWT



Final JWT =

xxxxx . yyyyy . zzzzz

1. Header (Meta Information)

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

- alg → Algorithm used for signing (e.g. HS256, RS256)
- typ → Type of token (always JWT)

2. Payload (Claims)

```
{
  "sub": "12345",
  "name": "Aman Saini",
  "role": "ADMIN",
  "iat": 1717000000,
  "exp": 1717003600
}
```

- sub → Subject (user id)
- name → User name
- role → User role
- iat → Issued At (timestamp)
- exp → Expiration time (timestamp)

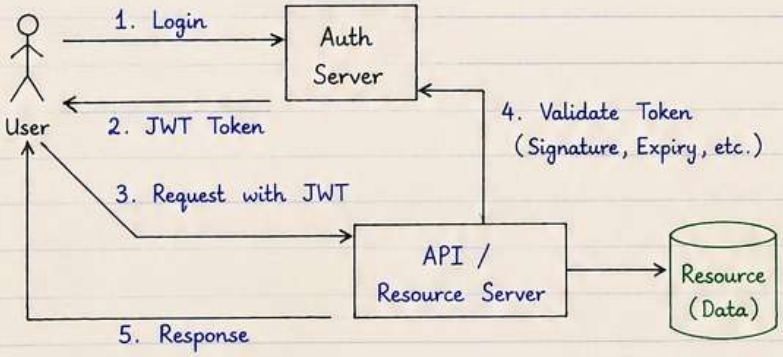
} These are known as Claims

3. Signature (Verification)

```
HMACSHA256( base64UrlEncode(header) + "." +
        base64UrlEncode(payload), secretKey )
```

It is used to verify that the token is not tampered and is from a trusted source.

JWT Authentication Flow



1. User logs in with credentials.
2. Auth server returns JWT.
3. User sends requests with JWT in Authorization header.
4. Server validates the token.
5. If valid → return data.

Example (Token in Authorization Header)

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJzdWIiOiIxMjM0NSIsIm5hbWIiOiJBbWFnIFNhaW5pIiwicm9sZSI6IkFETUlNIiwiaWF0IjoxNzE3...
8QyPbm7X1G9s... (signature)
```

5. OAUTH 2.0 (AUTHORIZATION FRAMEWORK)

What is OAuth 2.0 ?

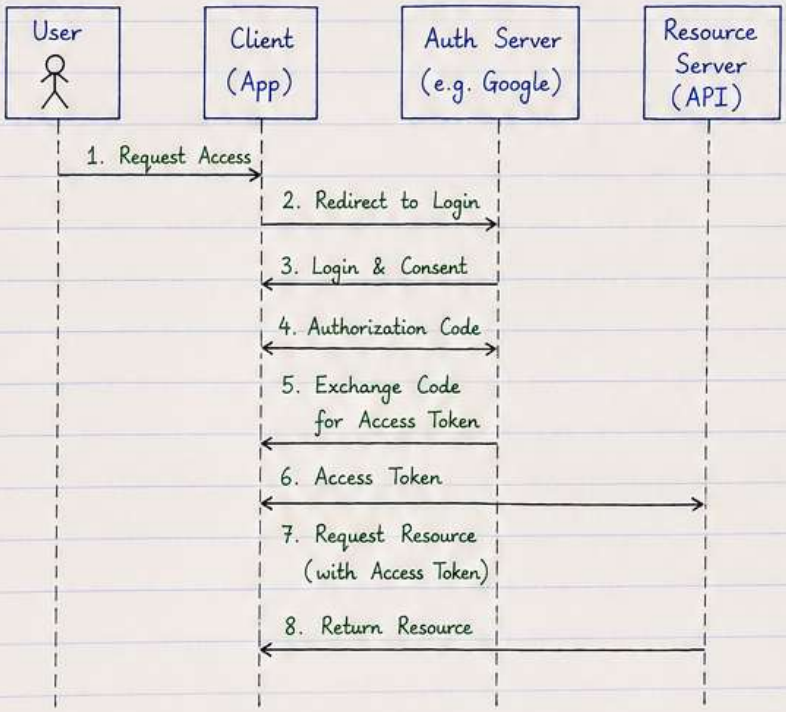
→ OAuth 2.0 is an authorization framework that allows third-party applications to access user resources on their behalf without sharing credentials.

→ It is not an authentication protocol.

Key Roles

- Resource Owner → The user
- Client → Application requesting access
- Authorization Server → Issues tokens
- Resource Server → Holds the protected resources (APIs)

OAuth 2.0 Flow (Authorization Code Grant)



1. User wants to access a resource through the client app.
2. Client redirects user to Authorization Server.
3. User logs in and grants consent.
4. Auth Server gives Authorization Code to client.
5. Client exchanges code for Access Token.
6. Auth Server returns Access Token.
7. Client calls Resource Server with Access Token.
8. Resource Server returns data.

Types of OAuth 2.0 Grants

1. Authorization Code Grant → Used by web & mobile apps (most secure)
2. Implicit Grant → Used by browser-based apps (token in redirect)
3. Client Credentials Grant → Used by server-to-server communication
4. Resource Owner Password Credentials Grant → User provides credentials to client
5. Refresh Token Grant → Used to get new access token without re-authentication

Access Token vs Refresh Token

	Access Token	Refresh Token
Purpose	Access protected resources	Get new access token
Lifetime	Short (minutes)	Long (days/weeks)
Usage	Sent with every API request	Used rarely (when token expires)
Risk	If leaked → limited damage	If leaked → high risk
Storage	In memory	Secure storage (DB / Keychain)

Example (Token Response)

```
{
  "access_token" : "eyJhbGciOiJIUzI1NiIs...",
  "token_type"   : "Bearer",
  "expires_in"   : 3600,
  "refresh_token": "dGhpcy-1pcy-yZWZyZXNo...",
  "scope"        : "read write",
}
```

Notes

- Always use HTTPS.
- Store tokens securely.
- Validate token (signature, expiry) on Resource Server.
- Use least privilege (scopes) for security.

6. DNS (DOMAIN NAME SERVER)

What is DNS ?

→ DNS (Domain Name System) is a distributed naming system that translates human readable domain names (e.g. www.example.com) into IP addresses (e.g. 93.184.216.34).

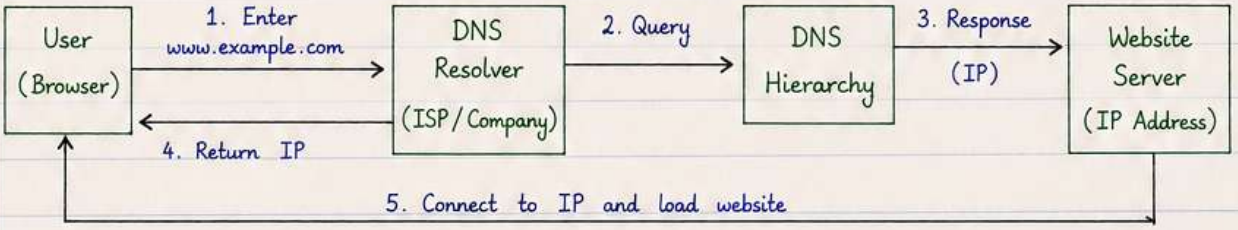
→ It works like the Internet's phonebook.

→ It is an application layer protocol that uses UDP / TCP (port 53).

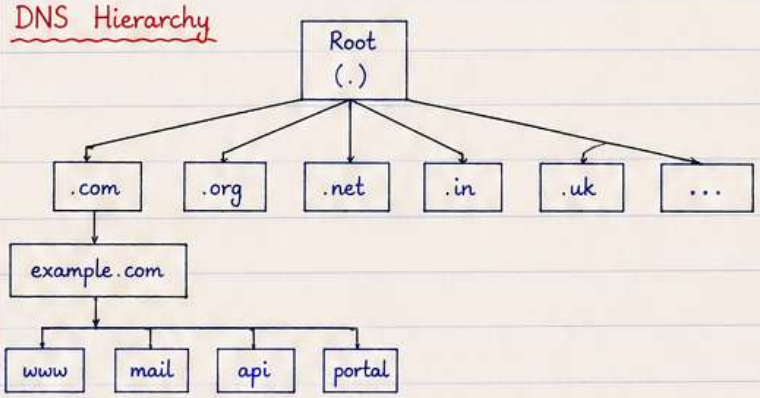
Why DNS is Needed ?

- Humans remember names, not IPs.
- IPs can change, names are easy to manage.
- Supports load balancing, redundancy and scalability.
- Helps in locating services across the Internet.

How DNS Works ? (High Level Flow)



DNS Hierarchy



- Root servers know all Top Level Domains (TLDs).
- TLD servers know the authoritative servers for each domain.
- Authoritative servers store the actual DNS records of the domain.

Common DNS Records

A	– Maps domain name to IPv4 address.
AAAA	– Maps domain name to IPv6 address.
CNAME	– Alias of another domain name.
MX	– Mail exchange server for the domain.
NS	– Name server (authoritative) for the domain.
TXT	– Text information (SPF, verification, etc.).

Example (A Record)

Name	Type	Value	TTL
example.com	A	93.184.216.34	300
www.example.com	A	93.184.216.34	300
mail.example.com	A	93.184.216.35	300

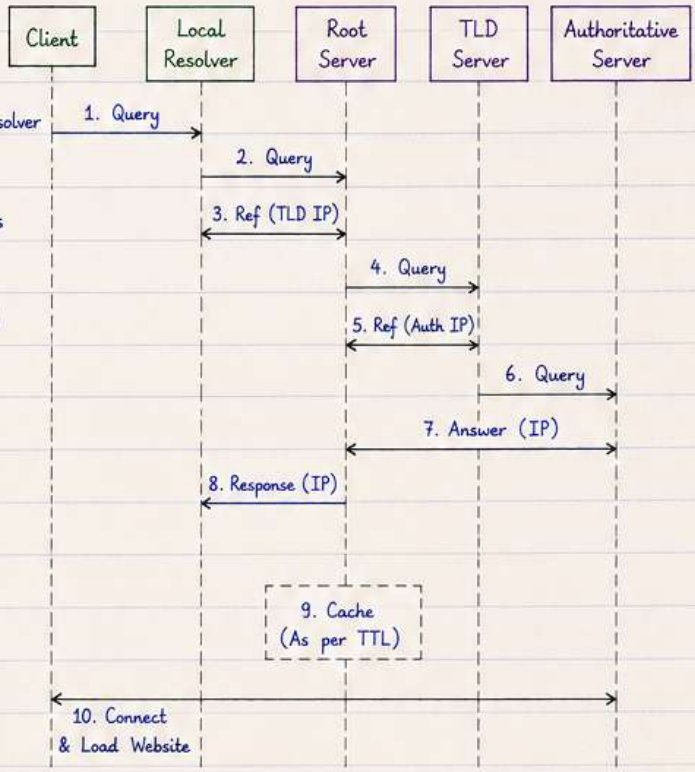
DNS Characteristics

- Distributed and hierarchical.
- Caching is used to improve performance.
- Highly available and fault tolerant.
- Requests are usually sent over UDP, TCP is used for large responses or zone transfers.

6. DNS (DOMAIN NAME SERVER) (2/2)

DNS Resolution Process (Detailed)

1. Browser checks cache (OS / Browser cache). If IP is found, it uses it.
2. If not found, the query is sent to the Local DNS Resolver (provided by ISP or configured).
3. If the resolver doesn't have the answer, it queries the Root Server.
4. Root Server returns the IP of the appropriate TLD server (.com, .org, .net, etc.).
5. Resolver queries the TLD server.
6. TLD server returns the IP of the authoritative name server for that domain.
7. Resolver queries the Authoritative Name Server.
8. Authoritative server returns the IP (DNS record).
9. Resolver caches the record (as per TTL) and returns the IP to the browser.
10. Browser connects to the IP and loads the website.



Types of DNS Records (Common)

Record Type	Purpose	Example	Example Value
A	Maps domain to IPv4 address	example.com	93.184.216.34
AAAA	Maps domain to IPv6 address	example.com	2001:0db8:85a3::8a2e:0370:7334
CNAME	Alias of another domain	www.example.com	example.com
MX	Mail exchange server	example.com	mail.example.com
NS	Name server (authoritative)	example.com	ns1.example.com
TXT	Text information (SPF, verification, etc.)	example.com	"v=spf1 include:_spf.example.com ~all"
PTR	Reverse lookup (IP to domain)	34.216.184.93.in-addr.arpa	example.com

DNS Caching and TTL

- TTL (Time To Live) defines how long a record can be cached.
- Higher TTL → Fewer DNS queries, but changes take longer to propagate.
- Lower TTL → More DNS queries, but changes are reflected quickly.

Example
If a record has TTL = 300 (5 minutes), resolvers can use the cached IP for 5 minutes before asking again.

Forward and Reverse Lookup

- Forward Lookup : Domain name → IP address (A/AAAA Record)
  Example : www.example.com → 93.184.216.34
- Reverse Lookup : IP address → Domain name (PTR Record)
  Example : 93.184.216.34 → www.example.com

DNS Security

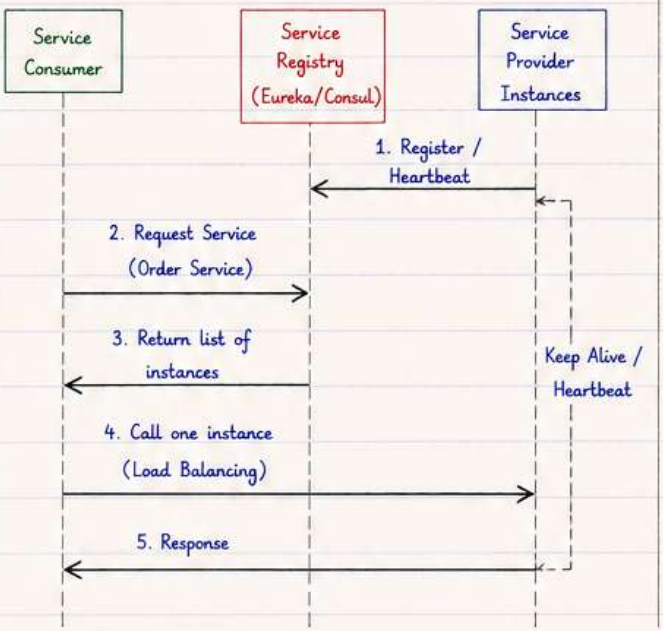
- DNS spoofing / cache poisoning can redirect users to malicious sites.
- DNSSEC (DNS Security Extensions) helps ensure data integrity and authenticity.
- Always use trusted DNS resolvers (e.g., 1.1.1.1, 8.8.8.8).

Common Public DNS

- Google DNS : 8.8.8.8, 8.8.4.4
- Cloudflare DNS : 1.1.1.1, 1.0.0.1
- Quad9 DNS : 9.9.9.9

7. SERVICE DISCOVERY (CONTINUED)

Service Discovery Flow (High Level)



How it Works ?

1. Service Provider starts and registers itself with the Service Registry with its host, port, and metadata.
2. Registry stores the details and makes the service available.
3. Service Consumer asks the Registry for the required service.
4. Registry returns the list of healthy instances.
5. Consumer selects one instance (client-side load balancing) and calls it.
6. Provider keeps sending heartbeats to the Registry. If heartbeats stop, it is removed (unhealthy).

Service Registry vs Service Discovery

Aspect	Service Registry	Service Discovery
Definition	Stores information about services and their instances.	Process of finding available services.
Role	Central database of services.	Used by consumers to locate services.
Example Tools	Eureka, Consul, Zookeeper	Eureka Client, Consul Client
Action	Register, store, update, remove instances.	Query, fetch, and choose instances.

Service Discovery Patterns

- Client-side Discovery – Client queries registry and does load balancing.
- Server-side Discovery – Client asks a smart proxy / load balancer which queries registry and routes the request.
- Hybrid – Combination of both client-side and server-side approaches.

Common Use Cases

- Microservices communication in dynamic environments.
- Scaling services up / down without hardcoding addresses.
- Rolling deployments (new instances added automatically).
- Fault tolerance (unhealthy instances removed automatically).

Example (Eureka Response)

```
{
  "application": {
    "name": "ORDER-SERVICE",
    "instance": [
      {"instanceId": "order-1", "hostName": "10.0.0.1", "port": 8080, "status": "UP"},
      {"instanceId": "order-2", "hostName": "10.0.0.2", "port": 8080, "status": "UP"}
    ]
  }
}
```

Tools for Service Discovery

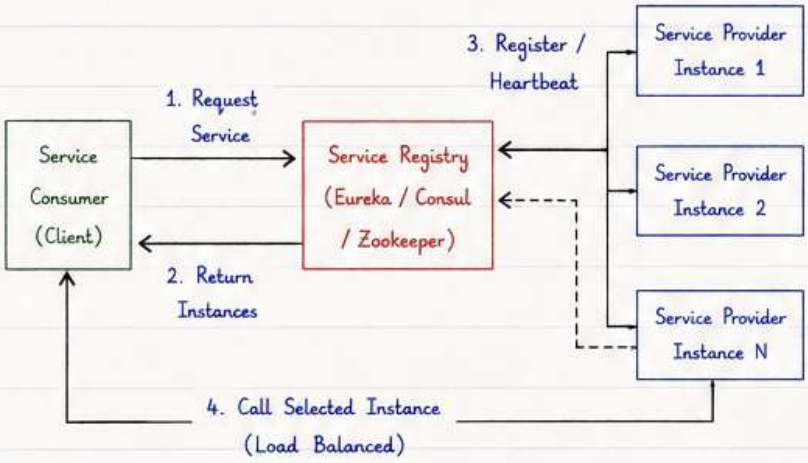
- Eureka – Netflix OSS, REST based, widely used with Spring Cloud.
- Consul – HashiCorp, supports service discovery, health check, key-value store.
- Zookeeper – Apache project, provides service discovery and coordination.
- etcd – Key-Value store with watch support, used in Kubernetes.

Key Points

- Never hardcode service URLs.
- Always use logical service names.
- Use health checks and timeouts.
- Combine with load balancing for high availability.
- Works great in containerized & cloud environments.

7. SERVICE DISCOVERY (CONTINUED) (2/2)

Typical Architecture



Service Discovery Lifecycle

1. Instance Start → Provider instance starts.
2. Register → It registers with the registry.
3. Heartbeat → Instance sends periodic heartbeats.
4. Healthy → If heartbeats are received, instance is marked healthy.
5. Unhealthy → If heartbeats stop, instance is removed.
6. Instance Stop → Instance deregisters or times out.

Service Discovery Tools Comparison

Tool	Type	Consistency	Key Features
Eureka	Netflix	Eventually Consistent	• REST based • Client side load balancing • Self preservation mode
Consul	HashiCorp	Strong Consistent	• Health checks • Key-Value store • Multi datacenter support
Zookeeper	Apache	Strong Consistent	• CP model • Watchers • High consistency
etcd	CNCF	Strong Consistent	• Key-Value store • Used in Kubernetes • Fast and reliable

Pros

- Decouples service consumers from providers.
- Supports dynamic scaling.
- Improves availability and fault tolerance.
- Enables rolling updates without downtime.
- Simplifies infrastructure management.

Cons

- Added complexity in the system.
- Extra network calls for discovery.
- Registry can become a single point of failure (if not HA).
- Misconfiguration can lead to outages.

Example Flow (Eureka)

Order Service instance starts. → Registers with Eureka Server. → Eureka stores instance info. → Payment Service requests for Order Service. → Eureka returns list of healthy instances. → Payment Service calls one instance using client-side load balancing.

Service Discovery Patterns

- Client-side Discovery – Client queries the registry and makes the call.
- Server-side Discovery – A proxy / load balancer queries the registry and routes the call.
- Health Check Pattern – Only healthy instances are returned by registry.
- Failover Pattern – If one instance fails, another healthy instance is used.

Best Practices

- Use multiple registry nodes for high availability.
- Configure proper health checks.
- Set appropriate timeouts and retry policies.
- Enable self-preservation mode (Eureka) in production.
- Monitor registry and service instances.
- Secure registry communication.

Real World Examples

8. PAYMENT SYSTEM DESIGN

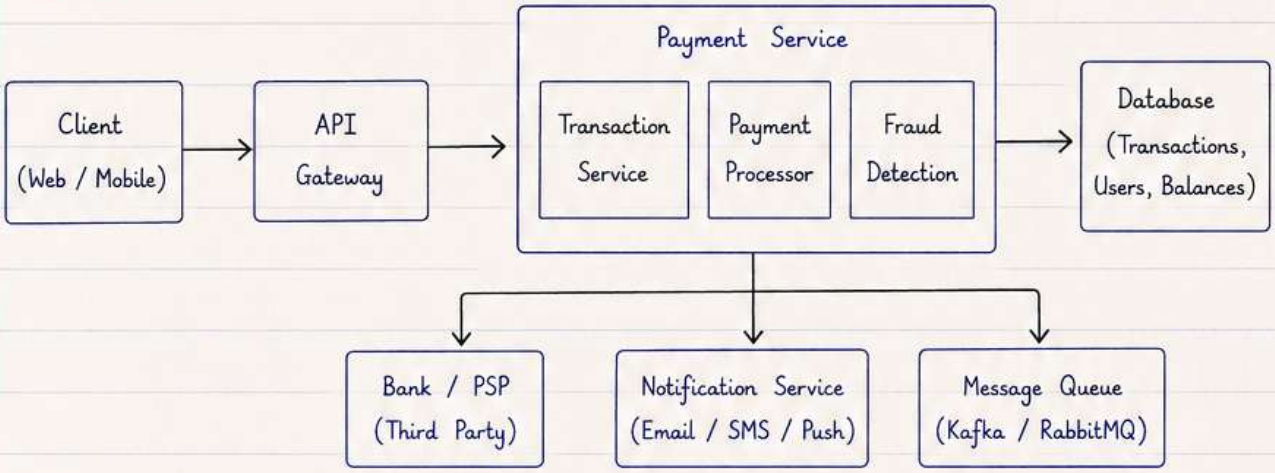
What is it ?

A payment system allows users to transfer money electronically from one party to another securely and reliably.

Goals

- Secure and reliable transactions
- High availability and scalability
- Low latency
- Accurate and consistent records

High Level Architecture



Key Components

- Client: Initiates the payment request.
- API Gateway: Handles routing, authentication, rate limiting.
- Transaction Service: Creates and manages transactions.
- Payment Processor: Integrates with banks or payment gateways to process payments.
- Fraud Detection: Detects and blocks suspicious transactions.
- Database: Stores users, accounts, and transaction data.
- Message Queue: Handles async events like payment status, retries.
- Notification Service: Sends receipts and alerts to users.

Payment Flow

1. User initiates payment on client.
2. Request is sent to API Gateway.
3. Payment Service validates the request and creates a transaction.
4. Payment Processor sends request to Bank / PSP to process the payment.
5. On success, transaction is updated in database and user is notified.

Key Considerations

Security	Scalability	Reliability	Consistency	Performance
Encrypt data, use TLS, tokenization, 2FA, PCI-DSS compliance.	Design stateless services, use load balancing and database sharding.	Retries, timeouts, circuit breakers, monitoring and alerts.	Use ACID transactions, idempotency, eventual consistency.	Optimize APIs, use caching, async processing, database indexing.

9. RETRY PATTERN

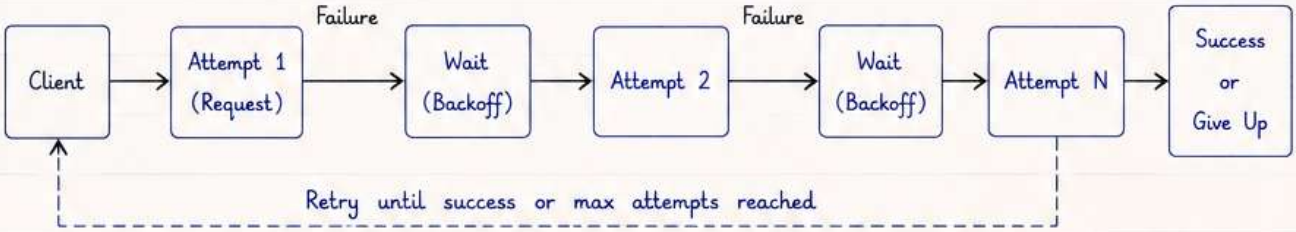
What is it ?

The Retry Pattern is used to handle temporary failures by retrying the operation before giving up.

When to use ?

- Network issues
- Temporary service unavailability
- Rate limiting (Too Many Requests)
- Transient errors (Timeouts, 5xx errors)

How it works



Key Points

- Retry only for transient errors.
- Use exponential backoff to avoid overloading the system.
- Set a maximum retry limit.
- Make retries idempotent to avoid duplicate side effects.
- Add jitter to backoff to reduce thundering herd problem.

Types of Retry

1. Immediate Retry – Retry immediately.
2. Fixed Delay Retry – Wait a fixed time between retries.
3. Exponential Backoff – Wait time increases exponentially (e.g., 1s, 2s, 4s, 8s...).
4. Exponential Backoff with Jitter – Add random jitter to wait time.

Example Flow (Exponential Backoff)

① Attempt 1 → Fail → Wait 1s
② Attempt 2 → Fail → Wait 2s
③ Attempt 3 → Fail → Wait 4s
④ Attempt 4 → Success → Stop

Max Retries = 4

Example Use Cases

- Calling external APIs
- Database connections
- Messaging systems
- File uploads / downloads

Best Practices

Practice	Why it matters
Retry only for transient errors	Avoid unnecessary retries for permanent failures.
Use exponential backoff	Prevents overwhelming the system.
Set max retry limit	Avoid infinite retries and save resources.
Add jitter	Reduces multiple clients retrying at the same time.
Ensure idempotency	Prevents duplicate operations on retry.

10. KEY VALUE STORE

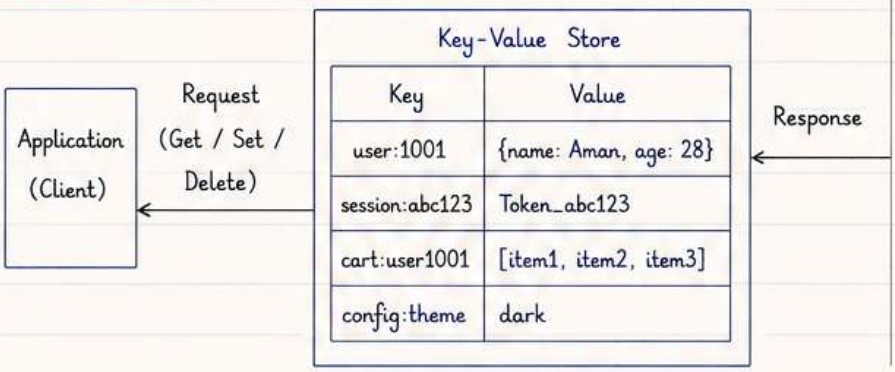
What is it ?

A Key-Value Store is a NoSQL database that stores data as a collection of key-value pairs.
It is simple, fast and highly scalable.

Key Characteristics

- Schema-less
- Fast read and write
- Horizontally scalable
- High availability
- Used for caching, sessions, preferences etc.

How it works



Basic Operations

- SET key value
  → Store a value with a key
- GET key
  → Retrieve value by key
- DELETE key
  → Remove key and its value

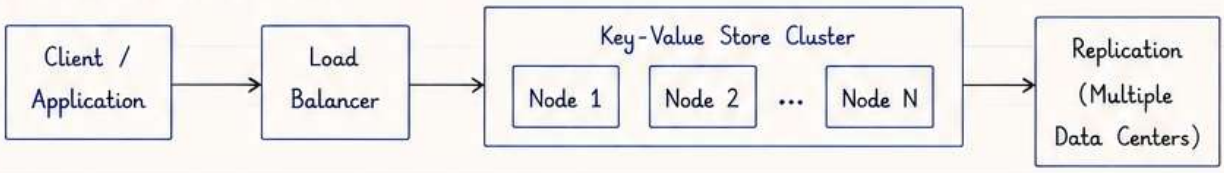
Use Cases

- Caching frequently accessed data
- User session store
- Shopping cart
- User preferences / Settings
- Real-time analytics, counters, leaderboards

Popular Examples

- Redis
- Amazon DynamoDB
- Riak
- Aerospike
- Amazon ElastiCache

Architecture (High Level)



Benefits

Benefit	Description
High Performance	Very fast read/write operations
Scalability	Easily scales horizontally
Simplicity	Simple data model (Key-Value)
Flexibility	Handles large amount of data of any type (string, list, hash etc.)

Limitations

Limitation	Description
No complex queries	Only key based lookups
Limited relationships	Not suitable for relational data
Memory usage	Mostly in-memory (cost can be high)
Persistence trade-off	Some systems may lose data if not configured properly

Key Points

- Best suited for simple lookups and high-performance use cases.
- Not ideal when you need complex queries or joins.
- Often used with caching and session management.

11. SCALE USERS FROM 0 TO MILLION (1/2)

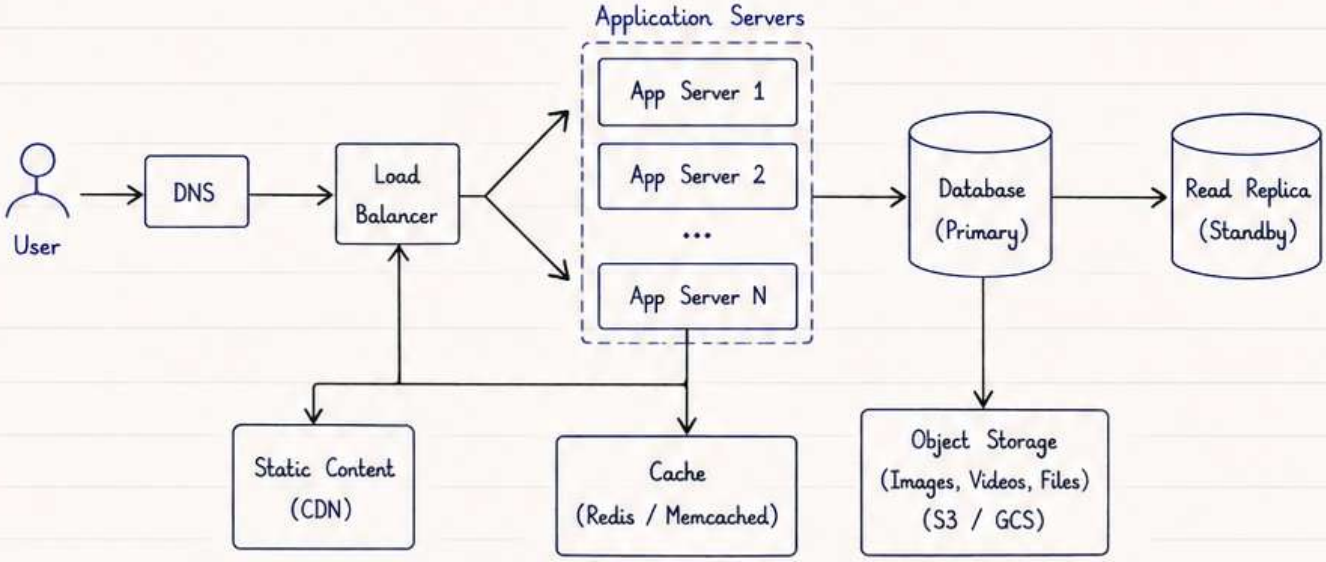
What is it ?

Design a system that can grow from zero users to millions of users while maintaining performance, reliability and low cost.

Key Challenges

- Handle increasing traffic
- Ensure high availability
- Store and serve large amount of data
- Keep system cost effective

High Level Architecture



Core Components

- DNS : Route users to closest server.
- Load Balancer : Distribute traffic across application servers.
- Application Servers : Handle user requests (stateless).
- Database : Store application data.
- Read Replica : Handle read traffic, reduce load on primary DB.
- Cache : Store frequently accessed data for fast response.
- CDN : Deliver static content (images, css, js) from edge locations.
- Object Storage : Store large files, media, backups.

Scaling Strategy (Overview)

- Start Small : Single server setup.
- Scale Up (Vertical) : Increase power of existing server when needed.
- Scale Out (Horizontal) : Add more servers behind load balancer.
- Database Scaling : Use read replicas, sharding when required.
- Caching : Add cache layer to reduce database load.
- CDN & Object Storage : Offload static and media content.

Key Principles

- Make systems stateless where possible.
- Separate read and write traffic.
- Automate scaling and deployments.

(Continued in next page →)

11. SCALE USERS FROM 0 TO MILLION (2/2)

Scaling Roadmap (Phases)

1. MVP
0 – 1K Users
- Monolith App
- Single Server
- Single DB

→

2. Early Growth
1K – 100K Users
- Load Balancer
- Multiple App Servers
- DB (Primary + Read Replica)

→

3. Scaling Up
100K – 1M Users
- Stateless Services
- Auto Scaling
- Cache Layer
- DB Sharding

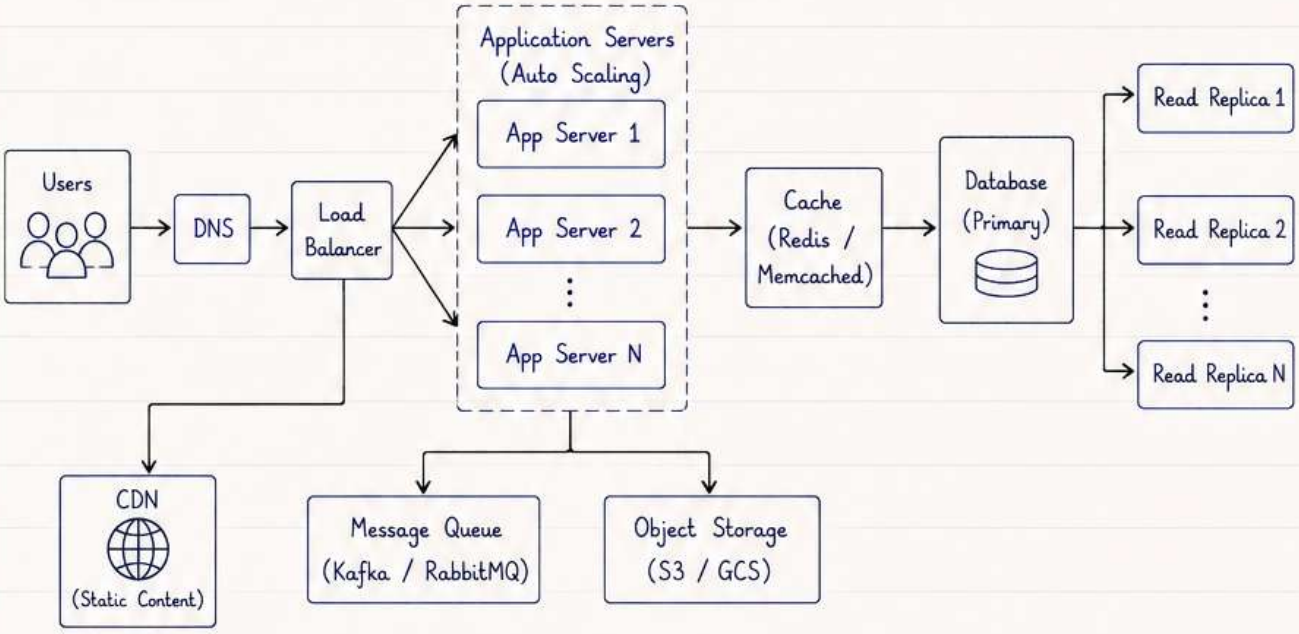
→

4. Large Scale
1M – 10M Users
- Microservices
- Multi-Region
- Message Queue
- CDN + Caching

→

5. Massive Scale
10M+ Users
- Global Scale
- Advanced Monitoring
- Cost Optimization

High Level Architecture (At Scale – 1M Users)



Key Strategies

- Distribute traffic using Load Balancer.
- Keep application stateless for easy scaling.
- Use Cache to reduce DB load.
- Database Read Replicas for heavy read traffic.
- Sharding to handle large write load.
- Offload static & media content to CDN / Object Storage.
- Use Message Queue for async processing.

Monitoring & Reliability

- Monitor metrics (CPU, Memory, RPS, Latency, Errors).
- Centralized logs and distributed tracing.
- Health checks and auto recovery.
- Data backup and disaster recovery.
- Alerts for anomalies and failures.

Things to Consider

- Plan for growth – design for scale from the beginning.
- Optimize database queries and indexing.
- Cost vs Performance trade-off.
- Security, rate limiting and abuse prevention.
- Regular load testing and capacity planning.

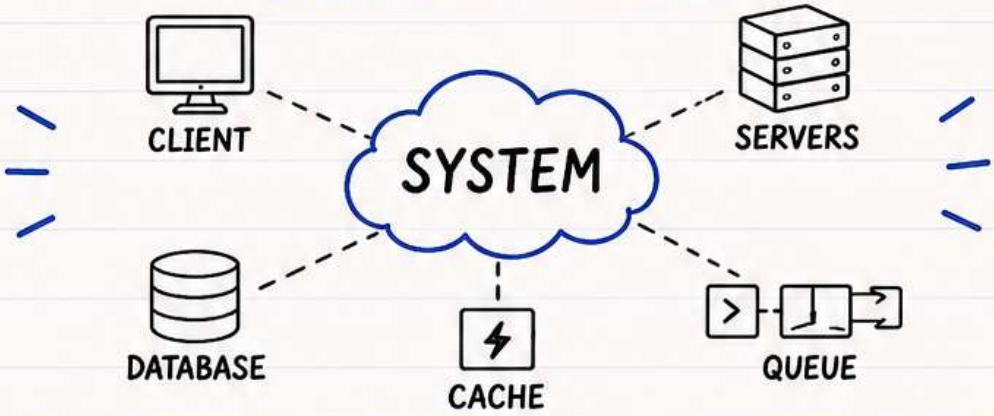
Outcome

A system that can handle millions of users with high availability, low latency, and great user experience, while keeping costs optimized.

FOLLOW
FOR
LLD
(LOW LEVEL DESIGN)

MORE ADVANCED TOPICS,
EXAMPLES & REAL-WORLD PROJECTS AHEAD!

COMMENT
SYSTEM DESIGN
FOR PDF NOTES



FOR LLD (LOW LEVEL DESIGN)
TARGET IS 10K